

K L Deemed to be University

DATA STRUCTURES AND ALGORITHMS - 2

CO2 Java Code - Segment Tree Range Queries

Course Outcome	CO2
Topic	Segment Trees
Language	Java
Document Type	Separate Implementation Code PDF

Objective

- Build a segment tree from an integer array for range-sum queries.
- Support point updates without rebuilding the entire array.
- Demonstrate $O(\log n)$ query and update behavior.

How to Run

Save the code in a file using the public class name shown in the program, compile it using `javac`, and run it using `java`.
Example: `javac FileName.java` followed by `java FileName`.

Complete Java Code

```
import java.util.*;

/*
 * CO2 Topic: Segment Tree
 * This program implements a range-sum segment tree with:
 * 1. O(n) build.
 * 2. O(log n) range sum query.
 * 3. O(log n) point update.
 */
public class CO2_SegmentTree_Demo {

    static class SegmentTree {
        int n;
        int[] tree;

        SegmentTree(int[] arr) {
            n = arr.length;
            tree = new int[4 * n];
            build(arr, 1, 0, n - 1);
        }

        private void build(int[] arr, int node, int left, int right) {
            if (left == right) {
                tree[node] = arr[left];
                return;
            }

            int mid = left + (right - left) / 2;
            build(arr, 2 * node, left, mid);
            build(arr, 2 * node + 1, mid + 1, right);
            tree[node] = tree[2 * node] + tree[2 * node + 1];
        }

        int rangeSum(int queryLeft, int queryRight) {
            return rangeSum(1, 0, n - 1, queryLeft, queryRight);
        }

        private int rangeSum(int node, int left, int right,
                             int queryLeft, int queryRight) {
            // No overlap
            if (queryRight < left || right < queryLeft) return 0;

            // Complete overlap
            if (queryLeft <= left && right <= queryRight) return tree[node];

            // Partial overlap
            int mid = left + (right - left) / 2;
            int leftSum = rangeSum(2 * node, left, mid, queryLeft, queryRight);
            int rightSum = rangeSum(2 * node + 1, mid + 1, right,
                                    queryLeft, queryRight);
            return leftSum + rightSum;
        }

        void update(int index, int newValue) {
            update(1, 0, n - 1, index, newValue);
        }

        private void update(int node, int left, int right,
                             int index, int newValue) {
            if (left == right) {
                tree[node] = newValue;
                return;
            }

            int mid = left + (right - left) / 2;
            if (index <= mid) update(2 * node, left, mid, index, newValue);
            else update(2 * node + 1, mid + 1, right, index, newValue);

            tree[node] = tree[2 * node] + tree[2 * node + 1];
        }

        void printImportantNodes() {
            System.out.println("Root sum [0," + (n - 1) + "] = " + tree[1]);
            if (n >= 8) {
                System.out.println("[0,3] sum = " + tree[2]);
                System.out.println("[4,7] sum = " + tree[3]);
                System.out.println("[0,1] sum = " + tree[4]);
                System.out.println("[2,3] sum = " + tree[5]);
                System.out.println("[4,5] sum = " + tree[6]);
                System.out.println("[6,7] sum = " + tree[7]);
            }
        }
    }
}
```

```
public static void main(String[] args) {
    int[] rackLoad = {12, 7, 9, 15, 6, 11, 10, 8};
    SegmentTree st = new SegmentTree(rackLoad);

    System.out.println("Initial segment tree summary:");
    st.printImportantNodes();

    int q1 = st.rangeSum(2, 6);
    int q2 = st.rangeSum(0, 3);
    System.out.println("sum(2, 6) before update = " + q1);
    System.out.println("sum(0, 3) before update = " + q2);

    System.out.println("Updating index 3 from 15 to 4...");
    st.update(3, 4);

    int q3 = st.rangeSum(2, 6);
    System.out.println("sum(2, 6) after update = " + q3);

    System.out.println("Updated segment tree summary:");
    st.printImportantNodes();
}
```

Expected Sample Output

```
Initial segment tree summary:
Root sum [0,7] = 78
[0,3] sum = 43
[4,7] sum = 35
[0,1] sum = 19
[2,3] sum = 24
[4,5] sum = 17
[6,7] sum = 18
sum(2, 6) before update = 51
sum(0, 3) before update = 43
Updating index 3 from 15 to 4...
sum(2, 6) after update = 40
Updated segment tree summary:
Root sum [0,7] = 67
[0,3] sum = 32
[4,7] sum = 35
[0,1] sum = 19
[2,3] sum = 13
[4,5] sum = 17
[6,7] sum = 18
```

Implementation Notes

- Segment tree nodes store information for intervals of the original array.
- Range query works by combining total-overlap nodes and ignoring no-overlap nodes.
- Point update changes one leaf and recomputes only the ancestors of that leaf.